

Exerciții C# — Metode, Validări și CRUD

Nivel începător+ · clase noi cu metode, `bool` de validare, login, operații pe listă

mycodeschool

Pentru: Ana

Ce e nou: metode care *decid* și metode care *schimbă starea*

Până acum metodele tale doar **descriau** un obiect (`Descriere()`) sau **calculau** o valoare (`ValoareStoc()`, `IsInStoc()`). Ai scris deja o metodă care **decide**:

```
public Boolean IsValidCredentials(String username, String password)
{
    if (this.nume.Equals(username) && this.password.Equals(password))
    {
        return true;
    }
    return false;
}
```

Aici sunt **trei idei** pe care le folosim în tot materialul:

1. **Metodă cu parametri** — primește date din afară (`username, password`).
2. **Metodă de validare** — întoarce `bool` (`true / false`), ca să o pui direct într-un `if`.
3. **this** — diferențiază câmpul obiectului (`this.password`) de parametru (`password`).

Mai adăugăm una: metode care **modifică obiectul** (schimbă un câmp) — de tip `void`, nu întorc nimic, dar lasă obiectul schimbat.

```
public void Depune(double suma)    // void = nu returnează, dar schimbă soldul
{
    this.sold = this.sold + suma;
}
```

Regula de aur: dacă metoda **răspunde la o întrebare** → întoarce `bool`. Dacă **face o acțiune** pe obiect → e `void`. Dacă **calculează ceva** → întoarce `double / int / string`.

Partea 1 — Metode de acțiune cu validare

Exercițiul 1 — Clasa `ContBancar`: depunere

Creează o clasă nouă `ContBancar` cu câmpurile: `titular` (string) și `sold` (double). Adaugă o metodă `Depune(double suma)` care **mărește soldul** cu suma primită — dar **doar dacă** suma e pozitivă. Dacă suma e ≤ 0 , afișează un mesaj și nu modifica nimic.

Exemplu: cont cu `sold = 100`, apoi `Depune(50)` → sold devine 150. `Depune(-20)` → mesaj „Suma trebuie să fie pozitivă” și sold rămâne 150.

Metoda e void (schimbă starea, nu returnează). Validarea e un `if` la început.

Exercițiul 2 — ContBancar: retragere cu două condiții

Adaugă metoda `Retrage(double suma)`. Poți retrage **doar dacă**: suma e pozitivă **și** ai destui bani (`suma <= sold`). Dacă merge, scazi din sold. Dacă nu, afișezi de ce nu (fonduri insuficiente / sumă invalidă).

Exemplu: `sold = 150`. `Retrage(60)` → `sold 90`. `Retrage(200)` → mesaj „Fonduri insuficiente”, `sold` rămâne 90.

Două condiții legate cu `&&`. Gândește-te ce mesaj dai pe fiecare caz greșit.

Exercițiul 3 — PoateRetrage întoarce bool

Adaugă o metodă `PoateRetrage(double suma)` care **nu modifică** soldul — doar **răspunde** cu `true` / `false` dacă retragerea ar fi posibilă. Apoi folosește-o într-un `if` în `Main`.

Exemplu: `sold = 90`. `PoateRetrage(50) → true`. `PoateRetrage(500) → false`.

Asta e diferența cheie: `Retrage` **face**, `PoateRetrage` **verifică**. O metodă `bool` se citește frumos:
`if (cont.PoateRetrage(50)) { ... }`.

Partea 2 — Login și validări pe câmpuri

Exercițiul 4 — Clasa Utilizator: login complet

Creează `Utilizator` cu: `username`, `parola`, `esteActiv` (bool). Scrie o metodă `Login(string username, string parola)` care întoarce `true` doar dacă `username`-ul și `parola` coincid și contul e activ (`esteActiv == true`).

Exemplu: user "ana" / parola "1234" / activ. `Login("ana", "1234")` → `true`. `Login("ana", "gresit")` → `false`. Cont inactiv → `false` chiar cu parola bună.

E `IsValidCredentials` + o condiție în plus. Trei lucruri legate cu `&&`.

Exercițiul 5 — Validare parolă puternică

Adaugă în `Utilizator` o metodă `ParolaEstePuternica()` care întoarce `true` dacă parola are **cel puțin 8 caractere**. Folosește `this.parola.Length`.

Exemplu: parola "1234" → `false`. Parola "parolaBuna1" → `true`.

`Length` îți dă numărul de caractere dintr-un string. E o metodă de validare „pură” — nu schimbă nimic, doar răspunde.

Exercițiul 6 — Validare email

Presupunem că `Utilizator` are și un câmp `email`. Scrie `EmailEsteValid()` care întoarce `true` dacă email-ul conține caracterul `@`.

Exemplu: `"ana@gmail.com" → true`. `"ana.gmail.com" → false`.

Indiciu: `this.email.Contains("@")` întoarce deja un `bool`. Poți chiar `return this.email.Contains("@");` direct.

Exercițiul 7 — Cont care se poate crea doar cu date valide

În `Main`, creează un `Utilizator`, setează-i câmpurile, apoi **înainte** de a-l folosi verifică-l: dacă `ParolaEstePuternica()` e `false` sau `EmailEsteValid()` e `false`, afișează „Date invalide, cont respins”. Altfel „Cont creat cu succes”.

Exemplu: parola scurtă → „Date invalide”. Parolă lungă + email cu @ → „succes”.

Aici combini două metode `bool` cu `||` (sau). E primul pas spre validarea reală dintr-un formular de înregistrare.

Partea 3 — CRUD pe o listă (Create, Read, Update, Delete)

CRUD = cele 4 operații pe care le face **orice** aplicație reală cu o listă de date: **Create** (adaugi), **Read** (cauți/afیzezi), **Update** (modifici), **Delete** (ștergi). Lucrăm pe `List<Produs>`.

Exercițiul 8 — CREATE: adaugă cu validare

Pornește de la o `List<Produs>` goală. Scrie o secvență care adaugă un produs în listă **doar dacă** numele nu e gol și prețul e > 0 . Dacă nu trece validarea, afیsează de ce nu s-a adăugat.

Exemplu: produs "laptop", preț 2500 $\rightarrow$ se adaugă (lista are 1 element). Produs cu nume gol $\rightarrow$ respins, lista rămâne goală.

Poți verifica numele gol cu `nume == ""` sau `string.IsNullOrEmpty(nume)`.

Exercițiul 9 — READ: caută un produs după nume

Ai o `List<Produs>` cu câteva produse. Scrie o secvență (sau o metodă `CautaProdus`) care parcurge lista și **întoarce produsul** cu numele căutat. Dacă nu-l găsește, întoarce `null`. Apoi în `Main` afișează produsul găsit sau „Nu există”.

Exemplu: cauți `"lanterna"` → afișezi descrierea ei. Cauți `"xyz"` → „Nu există”.

`null` = „niciun obiect”. După căutare verifici: `if (gasit != null) { ... }`. O metodă poate întoarce un obiect: tipul de retur devine `Produs`.

Exercițiul 10 — UPDATE: modifică prețul unui produs

Caută în listă produsul după nume (ca la ex. 9) și, dacă îl găsești, **schimbă-i prețul** la o valoare nouă. Afișează prețul înainte și după.

Exemplu: "laptop" de la 2500 $\rightarrow$ îl setezi la 2200. Verifici că `produs.pret` chiar s-a schimbat în listă.

Cheia de înțeles: dacă schimbi câmpul pe obiectul găsit, se schimbă **și în listă** — e **același** obiect, nu o copie. Țasta e conceptul „referință”.

Exercițiul 11 — DELETE: șterge un produs

Șterge din listă produsul cu un anumit nume. Poți folosi indexul din parcurgere și `produse.RemoveAt(i)`, sau `produse.Remove(produsGasit)`. Afișează `produse.Count` înainte și după.

Exemplu: lista are 3 produse, ștergi "bec" → rămân 2.

Atenție când ștergi în timpul unui `for` — dacă ștergi elementul `i`, elementele se decalează. Cel mai simplu: găsești întâi obiectul, apoi `Remove(obiect)`.

Exercițiul 12 — READ avansat: filtrează produsele în ofertă

Parcurge lista și afișează **doar** produsele care sunt `inOferta`. Numără și câte sunt. Folosește metoda `Descriere()` pentru afișare.

Exemplu: din 4 produse, 2 în ofertă → le afișezi pe cele 2 + „Total în ofertă: 2”.

Ai făcut deja filtrarea la exercițiile cu liste — aici doar o combini cu o metodă a obiectului.
Tiparul: `for + if (produse[i].inOferta)`.

Partea 4 — Aproape de realitate: mini-aplicații

Exercițiul 13 — Sistem de login pe o listă de utilizatori

Creează o `List<Utilizator>` cu 3 conturi. Scrie o secvență care: primește un `username` și o `parola` (le poți pune în variabile), **caută** utilizatorul cu acel `username` în listă, iar dacă îl găsește, apelează `Login(...)` pe el. Afășează „Autentificare reușită” / „Utilizator sau parolă greșite”.

Exemplu: listă cu `ana`, `raul`, `mihai`. Cauți `raul` + parola corectă → „reușită”. `raul` + parola greșită → „greșite”. `xyz` → „greșite” (nu există).

E CRUD-READ (cauți în listă) + validare (metoda `Login`) puse cap la cap. Exact așa funcționează un login real: întâi găsești contul, apoi verifici parola.

Exercițiul 14 — Coș de cumpărături

Creează o `List<Produs>` ca un „coș”. Scribe logica pentru:

1. **totalul** coșului (suma `pret * stoc = ValoareStoc()` pe fiecare);
2. o metodă `AplicaReducere(double procent)` pe `Produs` care **scade** din preț procentul dat (ex. 10%
→ `pret = pret - pret * 10 / 100`);
3. aplică 10% reducere pe tot coșul (parcurgi + apelezi metoda), apoi recalculează totalul.

Exemplu: total înainte 85, după 10% reducere pe fiecare → total mai mic.

Aici combini: metodă care **modifică** obiectul (`AplicaReducere, void`) + parcurgere + acumulator (totalul). E un mini-scenariu de magazin.

Exercițiul 15 (provocare) — CRUD complet într-un „meniu”

Pune la un loc tot: o `List<Produs>` și 4 acțiuni pe care le apelezi pe rând din `Main`, fiecare cu `Console.WriteLine` care spune ce s-a întâmplat:

- **Adaugă** 3 produse (cu validare, ex. 8).
- **Afișează** toată lista (Read, cu `Descriere()`).
- **Modifică** prețul unuia (Update, ex. 10).
- **Șterge** unul (Delete, ex. 11).
- **Afișează** lista finală.

Nu ai nevoie de meniu interactiv cu tastatura — doar apelează acțiunile în ordine și urmărește cum se schimbă lista. Țasta e scheletul oricărei aplicații de gestiune (produse, studenți, useri).

Recapitulare

Vreau să...	Tipul metodei	Cum arată
răspund la o întrebare	<code>bool</code>	<code>return conditie;</code> — o pui într-un <code>if</code>
schimb un câmp al obiectului	<code>void</code>	<code>this.camp = valoareNoua;</code>
calculez ceva	<code>double / int / string</code>	<code>return rezultat;</code>
găsesc un obiect în listă	întoarce obiectul sau <code>null</code>	parcurgi, <code>return produse[i];</code>
validez înainte de acțiune	<code>bool + if</code> la început	<code>if (!valid) return;</code>

Cele 4 operații CRUD pe o listă:

Operație	Cum
Create	validezi datele, apoi <code>lista.Add(obiect)</code>
Read	parcurgi cu <code>for / foreach</code> ; cauți după un câmp și întorci obiectul
Update	găsești obiectul, îi schimbi un câmp (se schimbă și în listă — referință)
Delete	găsești obiectul, <code>lista.Remove(obiect)</code>

Dacă stăpânești tabelul ăsta, ai scheletul oricărei aplicații reale de gestiune. Restul e doar câmpuri și reguli de validare diferite.